

SQL Developer Interview Guide



Rajat Saxena

(SENIOR ANALYST)

SQL Developer Interview Guide

-By Rajat Saxena

Question 1: What is SQL and what is it used for?

Answer: SQL (Structured Query Language) is a standard programming language used for managing and manipulating relational databases. It's used to perform tasks like querying data, inserting new data, updating existing data, deleting data, and creating/modifying database structures.

Question 2: What are the main types of SQL commands?

Answer: The main types of SQL commands are:

- **DDL (Data Definition Language):** Used for defining and managing database structures (e.g., CREATE, ALTER, DROP).
 - **DML (Data Manipulation Language):** Used for manipulating data within tables (e.g., SELECT, INSERT, UPDATE, DELETE).
 - **DCL (Data Control Language):** Used for managing permissions and access (e.g., GRANT, REVOKE).
 - **TCL (Transaction Control Language):** Used for managing transactions (e.g., COMMIT, ROLLBACK, SAVEPOINT).
-

Question 3: Explain the difference between DELETE and TRUNCATE statements.

Answer:

DELETE: Is a DML command. It removes rows one by one based on a condition (or all rows if no condition is specified). It logs each row deletion, which can be rolled back. It fires DELETE triggers.

TRUNCATE: Is a DDL command. It removes all rows from a table quickly by deallocating the data pages. It cannot be rolled back easily (depending on the RDBMS) and does not fire DELETE triggers. It is generally faster for large tables.

Question 4: What is a primary key?

Answer: A primary key is a column or a set of columns that uniquely identifies each row in a table. It must contain unique values and cannot contain NULL values. It enforces entity integrity.

Question 5: What is a foreign key?

Answer: A foreign key is a column or a set of columns in one table that refers to the primary key in another table. It establishes a link between two tables and enforces referential integrity, ensuring that relationships between tables remain consistent.

Question 6: What is the difference between INNER JOIN and LEFT JOIN?

Answer:

INNER JOIN: Returns only the rows where the join condition is met in *both* tables.

LEFT JOIN: Returns all rows from the left table, and the matched rows from the right table. If there is no match, the result is NULL on the right side.

Question 7: What is GROUP BY used for?

Answer: The GROUP BY clause is used to group rows that have the same values in one or more columns into a summary row. It's typically used with aggregate functions (like COUNT, MAX, MIN, SUM, AVG) to perform calculations on each group.

Question 8: What is the purpose of the HAVING clause?

Answer: The HAVING clause is used to filter groups based on a specified condition. It works similarly to the WHERE clause, but HAVING is applied *after* the GROUP BY clause has grouped the rows, allowing you to filter based on aggregate function results.

Question 9: Explain NULL in SQL.

Answer: NULL represents a missing or unknown value in a database column. It is not the same as zero or an empty string. Comparisons with NULL (except IS NULL and IS NOT NULL) always result in UNKNOWN (which is treated as false in WHERE clauses).

Question 10: What is a subquery?

Answer: A subquery (or inner query or nested query) is a query embedded within another SQL query. It can be used in SELECT, FROM, WHERE, or HAVING clauses to return data that will be used by the outer query.

Question 11: What are aggregate functions? Give examples.

Answer: Aggregate functions perform a calculation on a set of values and return a single value. Examples include:

COUNT(): Counts the number of rows.

SUM(): Calculates the sum of values.

AVG(): Calculates the average of values.

MAX(): Finds the maximum value.

MIN(): Finds the minimum value.

Question 12: What is the difference between WHERE and HAVING?

Answer:

WHERE filters individual rows *before* they are grouped by GROUP BY.

HAVING filters groups *after* they have been created by GROUP BY, typically based on aggregate function results.

Question 13: What is a VIEW in SQL?

Answer: A view is a virtual table based on the result-set of an SQL statement. It contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database. It doesn't store data itself but presents data from underlying tables.

Question 14: What are the different types of JOINS?

Answer: The main types of JOINS are:

- **INNER JOIN:** Returns matching rows from both tables.
 - **LEFT JOIN (or LEFT OUTER JOIN):** Returns all rows from the left table and matched rows from the right table.
 - **RIGHT JOIN (or RIGHT OUTER JOIN):** Returns all rows from the right table and matched rows from the left table.
 - **FULL JOIN (or FULL OUTER JOIN):** Returns all rows when there is a match in either the left or right table.
 - **CROSS JOIN:** Returns the Cartesian product of rows from both tables (all possible combinations).
-

Question 15: What is UNION and UNION ALL? What's the difference?

Answer: Both UNION and UNION ALL are used to combine the result sets of two or more SELECT statements.

UNION: Combines the results and removes duplicate rows.

UNION ALL: Combines the results and includes all rows, even duplicates. UNION ALL is generally faster as it doesn't need to check for duplicates.

Question 16: Explain the DISTINCT keyword.

Answer: The DISTINCT keyword is used in a SELECT statement to return only unique values. It eliminates duplicate rows from the result set.

Question 17: What is an index in SQL? Why is it useful?

Answer: An index is a data structure that improves the speed of data retrieval operations on a database table. It works like an index in a book, allowing the database to quickly locate rows without scanning the entire table. Indexes are useful for speeding up SELECT queries, WHERE clauses, and JOIN operations.

Question 18: What are VARCHAR and CHAR data types?

Answer:

CHAR(n): A fixed-length string. If the string is shorter than n, it's padded with spaces. If it's longer, it might be truncated or cause an error (depending on RDBMS).

VARCHAR(n): A variable-length string. It stores only the characters entered plus a small overhead for length. It's more efficient in terms of storage if string lengths vary significantly.

Question 19: What is normalization? What are its goals?

Answer: Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity.

Its main goals are:

- Minimizing data duplication.
- Ensuring data dependencies make sense (i.e., data is stored in the logically correct table).

- Simplifying database structure, making it easier to maintain and update.

Question 20: What is a schema in SQL?

Answer: A schema is a collection of database objects (like tables, views, stored procedures, functions) that are owned by a particular user or role. It acts as a namespace to organize database objects and control access.

Question 21: Explain the concept of a LEFT OUTER JOIN with an example.

Answer: A LEFT OUTER JOIN returns all records from the left table (the table mentioned before LEFT JOIN), and the matched records from the right table. If there is no match for a row in the left table, the result will contain NULL values for columns from the right table.

```
Example: SELECT Customers.CustomerName, Orders.OrderID  
FROM Customers  
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

This query will list all customer names, and for customers who have placed orders, it will also show their OrderID. Customers who haven't placed any orders will still be listed, but their OrderID will be NULL.

Question 22: What are Common Table Expressions (CTEs)? When are they useful?

Answer: A Common Table Expression (CTE) is a temporary, named result set that you can reference within a single SQL statement (like SELECT, INSERT, UPDATE, or DELETE).

They are useful for:

- Breaking down complex queries into simpler, more readable logical units.
- Performing recursive queries (e.g., hierarchical data).

- Avoiding the need for temporary tables or complex subqueries.

Question 23: Write a SQL query to find the second highest salary from an Employees table with Salary and EmployeeID columns.

Answer: There are several ways to do this, here's one using a subquery and LIMIT:

```
SELECT Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 1 OFFSET 1;
```

`` *(Note: `OFFSET 1` skips the first row, and `LIMIT 1` selects the next single row. Syntax might slightly vary based on the specific SQL dialect, e.g., `TOP` in SQL Server.)*

Question 24: What is a CASE statement in SQL? Provide a simple example.

Answer: A CASE statement allows you to write IF-THEN-ELSE logic within SQL queries. It evaluates a list of conditions and returns a value when the first condition is met.

```
Example: SELECT
  ProductName,
  CASE
    WHEN Price > 100 THEN 'Expensive'
    WHEN Price BETWEEN 50 AND 100 THEN 'Moderate'
    ELSE 'Cheap'
  END AS PriceCategory
FROM Products;
```

Question 25: Explain the difference between RANK(), DENSE_RANK(), and ROW_NUMBER() window functions.

Answer: These are all window functions used for ranking rows within a partition:

ROW_NUMBER(): Assigns a unique sequential integer to each row within its partition, starting from 1. No two rows will have the same row number.

RANK(): Assigns a rank to each row within its partition. If two rows have the same value, they receive the same rank, and the next rank is skipped (e.g., 1, 2, 2, 4).

DENSE_RANK(): Assigns a rank to each row within its partition. If two rows have the same value, they receive the same rank, and the next rank is *not* skipped (e.g., 1, 2, 2, 3).

Question 26: What is a stored procedure? What are its advantages?

Answer: A stored procedure is a precompiled collection of one or more SQL statements stored in the database.

Advantages:

- Performance: Compiled once, executed many times.
 - Reusability: Can be called from multiple applications.
 - Reduced Network Traffic: Only the procedure name and parameters are sent over the network, not the entire SQL statement.
 - Security: Can grant execute permissions without granting direct table access.
 - Maintainability: Logic is centralized.
-

Question 27: What is a trigger in SQL?

Answer: A trigger is a special type of stored procedure that automatically executes or fires in response to certain events on a particular table or view. These events are typically INSERT, UPDATE, or DELETE operations. Triggers are often used for enforcing business rules, auditing changes, or maintaining data integrity.

Question 28: What is an ad-hoc query?

Answer: An ad-hoc query is a query that is created and executed on the fly to retrieve specific information, often for analysis or investigation, rather than being part of a pre-defined application or report.

Question 29: How do you find duplicate records in a table?

Answer: You can find duplicate records using GROUP BY and HAVING clauses.

Example (finding duplicate email addresses in a Users table):

```
SELECT Email,  
COUNT(*) AS EmailCount  
FROM Users  
GROUP BY Email  
HAVING COUNT(*) > 1;
```

Question 30: What is transaction isolation level? Name a few.

Answer: Transaction isolation levels define the degree to which one transaction must remain isolated from the data modifications made by other concurrent transactions. Higher isolation levels provide more data consistency but can reduce concurrency.

Common levels include:

- **READ UNCOMMITTED**
 - **READ COMMITTED**
 - **REPEATABLE READ**
 - **SERIALIZABLE**
-

Question 31: What is the difference between OR and IN operators?

Answer: * **OR:** Used to combine multiple conditions, where at least one condition must be true for a row to be selected.

* **IN:** Allows you to specify multiple values in a WHERE clause. It's a shorthand for multiple OR conditions when checking if a column's value matches any value in a list.

Example: WHERE City = 'London' OR City = 'Paris' is equivalent to WHERE City IN ('London', 'Paris').

Question 32: What is a composite key?

Answer: A composite key is a primary key that consists of two or more columns. Each column alone might not be unique, but the combination of their values uniquely identifies each row.

Question 33: What is a Self Join?

Answer: A self-join is a regular join, but the table is joined with itself. This is useful when you want to compare rows within the same table, such as finding employees who have the same manager or finding pairs of customers with similar addresses. You use table aliases to distinguish between the two instances of the table.

Question 34: Explain the COALESCE function.

Answer: The COALESCE function returns the first non-NULL expression from a list of expressions. It's useful for substituting NULL values with a default value.

Example:

```
SELECT  
    COALESCE(OrderDate, ShipDate, GETDATE()) AS EffectiveDate  
FROM Orders;
```

Explanation (simple):

- Returns OrderDate if it's NOT NULL
 - Otherwise returns ShipDate if it's NOT NULL
 - Otherwise returns the current date & time using GETDATE()
-

Question 35: What is a HAVING clause used for when GROUP BY is not used?

Answer: A HAVING clause is typically used with GROUP BY. If GROUP BY is not used, HAVING effectively acts like a WHERE clause but can also filter based on aggregate function results applied to the entire table (if aggregates are used). However, standard practice is to use WHERE for row-level filtering and HAVING for group-level filtering.

Question 36: What is a CURSOR in SQL? When should you avoid using it?

Answer: A cursor is a database object that allows you to traverse records in a result set one row at a time. It's like a pointer to a specific row in the result set. Cursors should generally be avoided because they are slow and inefficient compared to set-based operations. They are typically used only when row-by-row processing is unavoidable.

Question 37: What is database denormalization? When is it used?

Answer: Denormalization is the process of introducing redundancy into a normalized database design. It is typically done to improve read performance by reducing the number of JOIN operations required for frequent queries. It's often used in data warehousing or for applications with very high read loads where performance is critical, even at the cost of increased storage and complexity in writes.

Question 38: What is an EXISTS operator? How does it differ from IN?

Answer: The EXISTS operator is used to test for the existence of rows in a subquery. It returns TRUE if the subquery returns one or more records, and FALSE otherwise.

* **Difference from IN:** IN checks if a value is present in a list of values returned by a subquery. EXISTS checks if the subquery returns *any* rows at all. EXISTS is often more efficient

than IN when the subquery returns a large number of rows because it can stop processing as soon as it finds the first matching row.

Question 39: What are NULL handling functions in SQL?

Answer: These functions allow you to manage NULL values.

Common examples include:

- COALESCE(): Returns the first non-NULL expression.
 - ISNULL() (SQL Server specific): Replaces NULL with a specified value.
 - NVL() (Oracle specific): Similar to ISNULL.
 - IFNULL() (MySQL specific): Similar to ISNULL.
-

Question 40: Explain PIVOT and UNPIVOT operations.

Answer: * **PIVOT:** Rotates rows into columns. It transforms unique values from one row into multiple columns in the output.

* **UNPIVOT:** Rotates columns into rows. It transforms multiple columns in the input into multiple rows in the output. These operations are useful for restructuring data for reporting or analysis.

Question 41: What are Window Functions in SQL? Provide an example of ROW_NUMBER() used with PARTITION BY.

Answer: Window functions perform calculations across a set of table rows that are somehow related to the current row. They are similar to aggregate functions but do not collapse rows into a single output row. Instead, they return a value for each row. PARTITION BY divides the rows into partitions (groups), and the window function is applied independently to each partition.

Example: Assigning a rank to employees within each department based on salary.

```
SELECT
    EmployeeName,
    Department,
    Salary,
    ROW_NUMBER() OVER (
        PARTITION BY Department
        ORDER BY Salary DESC
    ) AS rn_within_dept
FROM Employees;
```

This query assigns a unique number to each employee within their department, with the highest salary getting rank 1.

Question 42: Explain the concept of DEFERRABLE constraints in SQL.

Answer: DEFERRABLE constraints allow the check of a constraint (like UNIQUE or FOREIGN KEY) to be delayed until the end of the transaction, rather than being checked immediately after each statement. This is useful in scenarios where you need to temporarily violate a constraint within a transaction but guarantee it will be valid by the time the transaction commits.

Question 43: What is a recursive CTE? Provide a scenario where it's useful.

Answer: A recursive CTE is a CTE that references itself. It's used to query hierarchical data, such as organizational charts, bill of materials, or graph structures.

A recursive CTE typically has two parts:

- 1. An anchor member:** This is the base query that returns the initial set of rows.
- 2. A recursive member:** This part references the CTE itself and is executed repeatedly, combining its results with the anchor member's results, until no more rows are returned.

Scenario: Displaying an employee hierarchy, starting from the CEO and listing all employees reporting to them, directly or indirectly.

Question 44: Explain ACID properties in database transactions.

Answer: ACID is an acronym representing four essential properties that guarantee reliable processing of database transactions:

- **Atomicity:** Ensures that a transaction is treated as a single, indivisible unit. Either all operations within the transaction are completed successfully, or none of them are.
 - **Consistency:** Guarantees that a transaction brings the database from one valid state to another. It ensures that data integrity constraints are maintained.
 - **Isolation:** Ensures that concurrent transactions do not interfere with each other. The outcome of concurrent transactions is the same as if they were executed serially.
 - **Durability:** Guarantees that once a transaction has been committed, its changes are permanent and will survive subsequent system failures (e.g., power outages or crashes).
-

Question 45: What is query optimization? How does the query optimizer work?

Answer: Query optimization is the process by which a database management system (DBMS) determines the most efficient way to execute a given SQL query. The query optimizer analyzes the query and considers various execution plans (e.g., different join orders, index usage, access methods). It uses statistics about the data (like table sizes, data distribution) to estimate the cost of each plan and chooses the one with the lowest estimated cost.

Question 46: What is a Clustered Index vs. a Non-Clustered Index?

Answer: * **Clustered Index:** Dictates the physical order of data rows in a table. A table can have only one clustered index. It's typically created on the primary key. Reading data sequentially according to the clustered index is very fast.

* **Non-Clustered Index:** A separate structure that contains the indexed columns and pointers to the actual data rows. The data rows themselves are not stored in the order of the non-clustered index. A table can have multiple non-clustered indexes.

Question 47: What are CTID (or equivalent) and when might you use it?

Answer: CTID (or similar identifiers like ROWID in Oracle, or TID in PostgreSQL) is a system-generated identifier that uniquely points to the physical location of a row within a table. It's not part of the SQL standard and its availability and behavior vary across RDBMSs.

It can be useful for:

- Quickly accessing or deleting specific rows if you know their physical location (though this is generally discouraged for production code due to potential changes after index rebuilds or VACUUM operations).
 - Debugging and understanding internal data storage.
-

Question 48: Explain the concept of ROWLOCK, PAGELOCK, and TABLOCK hints.

Answer: These are locking hints used in SQL Server to influence how the query optimizer acquires locks on data.

- **ROWLOCK:** The query optimizer acquires row-level locks.
 - **PAGELOCK:** The query optimizer acquires page-level locks.
 - **TABLOCK:** The query optimizer acquires a table-level lock. These hints can impact concurrency and performance, and should be used with caution after thorough testing.
-

Question 49: What is a phantom read? How can it be prevented?

Answer: A phantom read occurs when, during a transaction, two identical queries are executed, and the second query returns additional rows that were not returned by the first query. This typically happens when another transaction inserts new rows that satisfy the query's conditions between the first and second execution of the query within the same transaction.

* **Prevention:** Setting the transaction isolation level to **SERIALIZABLE** prevents phantom reads by ensuring that the execution of transactions is effectively serialized.

Question 50: What is the difference between UNION and UNION ALL in terms of performance and behavior?

Answer: * **Behavior:** UNION returns only distinct rows, removing duplicates. UNION ALL returns all rows, including duplicates.

* **Performance:** UNION ALL is generally faster because it doesn't have to perform the extra step of sorting and removing duplicate rows. UNION incurs overhead for duplicate detection. The choice depends on whether duplicate rows are acceptable in the final result.

Question 51: What are User-Defined Functions (UDFs)? Types and use cases.

Answer: UDFs are functions created by the user to perform specific calculations or operations. They can be used in SELECT, WHERE, JOIN clauses, etc.

Types:

- **Scalar Functions:** Return a single value.
- **Table-Valued Functions (TVFs):** Return a table (either inline or multi-statement).

Use Cases: Encapsulating complex logic, reusability, simplifying queries, performing custom data transformations.

Question 52: Explain the concept of "index fragmentation." What are its types and how to resolve it?

Answer: Index fragmentation occurs when the logical order of data in an index does not match its physical order on disk.

Types:

- **Internal Fragmentation:** Wasted space within index pages.
- **External Fragmentation:** Pages are not stored contiguously on disk.

Resolution: Reorganizing (defragmenting) or rebuilding the index. REORGANIZE attempts to fix fragmentation with minimal impact, while REBUILD creates a new index and is more thorough but also more resource-intensive.

Question 53: What are the different types of database normalization (1NF, 2NF, 3NF, BCNF)? Briefly explain them.

- **Answer: * 1NF (First Normal Form):** Each column must contain atomic values, and each row must be unique. No repeating groups.
 - **2NF (Second Normal Form):** Must be in 1NF, and all non-key attributes must be fully functionally dependent on the primary key. (Applies to tables with composite primary keys).
 - **3NF (Third Normal Form):** Must be in 2NF, and all non-key attributes must be non-transitively dependent on the primary key. (No transitive dependencies).
 - **BCNF (Boyce-Codd Normal Form):** A stricter version of 3NF. For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey.
-

Question 54: What is MERGE statement?

Answer: The MERGE statement (also known as UPSERT) is used to perform INSERT, UPDATE, or DELETE operations on a target table based on the results of joining it with a source table or query. It allows you to conditionally modify rows in the target table based on whether they match rows in the source.

Question 55: How does a FULL OUTER JOIN work?

Answer: A FULL OUTER JOIN returns all rows from both the left and right tables. If there's a match between the tables based on the join condition, the columns from both tables are combined in the result row. If a row in the left table has no match in the right table, the columns from the right table will contain NULL. Conversely, if a row in the right table has no match in the left table, the columns from the left table will contain NULL.

Question 56: What are Row-Level Security (RLS) and Column-Level Security (CLS)?

Answer: * Row-Level Security (RLS): Restricts access to specific rows in a table based on user credentials or other criteria. For example, a sales manager might only see sales data for their own region.

*** Column-Level Security (CLS):** Restricts access to specific columns in a table. For example, a user might see employee names and salaries but not sensitive personal information like social security numbers.

Question 57: Explain the difference between TRUNCATE, DELETE, and DROP.

- **Answer: TRUNCATE:** Removes all rows from a table but keeps the table structure. It's a DDL command, faster than DELETE, and typically cannot be rolled back.
 - **DELETE:** Removes rows from a table based on a condition or all rows if no condition is specified. It's a DML command, logs each row, can be rolled back, and fires triggers.
 - **DROP:** Removes an entire database object (like a table, index, or database) completely. It removes the object's structure and data and cannot be rolled back easily.
-

Question 58: What is a CHECKPOINT in SQL Server?

Answer: A checkpoint is a database operation that writes dirty pages (pages in memory that have been modified but not yet written to disk) from the buffer cache to disk and records the information about the transaction log. It serves as a recovery point, reducing the amount of log file that needs to be scanned during database startup or recovery.

Question 59: What is OLTP vs. OLAP?

Answer: OLTP (Online Transaction Processing): Systems designed for rapid, frequent, and small transactions (e.g., inserting an order, updating customer info). Focuses on data integrity, speed, and concurrency for operational tasks. Databases are typically highly normalized.

*** OLAP (Online Analytical Processing):** Systems designed for complex queries and analysis of large volumes of historical data (e.g., sales trends, financial reporting). Focuses on read performance and aggregation. Databases are often denormalized (e.g., star or snowflake schemas).

Question 60: What are the different types of NULL comparisons in SQL?

Answer: IS NULL: Checks if a value is NULL.

- **IS NOT NULL:** Checks if a value is not NULL.
- Standard comparison operators (=, !=, <, >) when used with NULL typically result in UNKNOWN, which is treated as false in WHERE clauses.

For example, WHERE column = NULL will not return any rows, even if column contains NULL.

Question 61: Explain the IIF function (if available in the specific RDBMS).

Answer: The IIF function is a shorthand for a simple CASE statement. It takes three arguments: a condition, a value to return if the condition is true, and a value to return if the condition is false.

```
SELECT
    ProductName,
    IIF(Price > 100, 'Expensive', 'Not Expensive') AS PriceLevel
FROM Products;
```

Question 62: What is a composite index? How is it different from a regular index?

Answer: A composite index (also called a multi-column index) is an index created on two or more columns of a table. The order of columns in the index definition matters. A composite index can be used to speed up queries that filter on the leading columns of the index. A regular (single-column) index is on just one column.

Question 63: What is the purpose of the SET NOCOUNT ON statement in SQL Server stored procedures?

Answer: SET NOCOUNT ON prevents SQL Server from sending the count of rows affected by a T-SQL statement back to the client. This can reduce network traffic and slightly improve performance, especially in stored procedures that execute many statements.

Question 64: What are the different types of database locks?

Answer: Database locks are used to manage concurrency and prevent data corruption.

Common types include:

- **Shared Locks (S):** Allow reading data but prevent modification.
 - **Exclusive Locks (X):** Allow reading and modification but prevent other transactions from acquiring any lock on the resource.
 - **Intent Locks (IS, IX, SIX):** Indicate the intention to acquire locks at lower granularities (e.g., row locks).
 - **Update Locks (U):** Used when reading data that might be updated later, preventing a deadlock situation where two transactions try to update the same row.
-

Question 65: Explain the concept of "Database Sharding."

Answer: Sharding is a database architecture technique where a large database is broken down into smaller, more manageable pieces called "shards." Each shard typically resides on a separate database server. Sharding is used to improve scalability, performance, and availability by distributing data and query load across multiple servers.

Question 66: What is a Stored Function vs. a Stored Procedure?

Answer: * **Stored Procedure:** A set of SQL statements stored in the database that performs an action. It does not return a value directly but can have output parameters.

* **Stored Function:** A set of SQL statements that returns a single value (scalar function) or a table (table-valued function). Functions can be called directly within SQL statements (e.g., in SELECT or WHERE clauses), whereas procedures are typically executed using EXECUTE or CALL.

Question 67: What is a composite primary key and when would you use it?

Answer: A composite primary key is a primary key that consists of two or more columns. It's used when a single column is not sufficient to uniquely identify each row in a table. You would use it when the combination of values across multiple columns is required to ensure uniqueness and represent a logical entity (e.g., in a junction table for a many-to-many relationship, where a combination of foreign keys uniquely identifies a record).

Question 68: What is a SELECT INTO statement used for?

Answer: The SELECT INTO statement is used to create a new table and insert the result of a SELECT query into it. It's a convenient way to copy data from one table to another or to create a backup of a table's structure and data. The syntax can vary slightly between RDBMSs (e.g., SELECT ... INTO ... in SQL Server, CREATE TABLE ... AS SELECT ... in PostgreSQL/Oracle).

Question 69: What is a "covering index"?

Answer: A covering index is a non-clustered index that includes all the columns required to satisfy a query (both in the SELECT list and WHERE clause). If a query can be fully satisfied by reading only the index (without needing to access the base table), it's called a covering index, and it can significantly improve query performance.

Question 70: What is a "deadlock"? How can it be detected and resolved?

Answer: A deadlock occurs when two or more transactions are each waiting for a resource that the other transaction holds, creating a circular dependency.

* **Detection:** Most database systems have mechanisms to automatically detect deadlocks.

* **Resolution:** The database system typically resolves a deadlock by choosing one of the deadlocked transactions as a "victim," rolling back that transaction, and releasing its locks so the other transaction(s) can proceed. Developers can help prevent deadlocks by ensuring consistent lock ordering (e.g., always locking tables/rows in the same order) and keeping transactions short.

Thank You for Reading

Thank you for taking the time to read the *SQL Developer Interview Guide*.

I truly hope this guide helps you strengthen your SQL concepts, gain confidence, and perform better in your interviews. This book was created with one simple goal — **to make interview preparation clearer, more practical, and less stressful for job seekers.**

Your learning journey doesn't have to stop here. If you need ongoing guidance, interview tips, job updates, or resume support, feel free to stay connected with me through the platforms below:

 [LinkedIn Profile](#)

 [WhatsApp Channel](#)

 [Telegram Channel](#)

I regularly share **job openings, interview questions, preparation strategies, and career advice** to help you move one step closer to your goals.

Wishing you all the very best for your interviews and career ahead.
Keep learning. Keep growing. And never stop believing in yourself.

Warm regards,

[Rajat Saxena](#)